FRAMES AND SCRIPTS

Submitted by
Nimisha .K
Roll no : 55
S8,CS-B

FRAMES

A frame is a collection of attributes (called slots) and associated values that describe some entity in the world.
Sometimes a frame describes an entity in some absolute sense, sometimes it represents the entity from a particular point of view only.
A single frame taken alone is rarely useful, we build frame systems out of collections of frames that are connected to each other by virtue of the fact that the value of an attribute of one frame may be another frame.

Frames as Sets and Instances

The set theory is a good basis for understanding frame systems. Each frame represents either a class (a set) or an instance (an element of class)

Example of frames

Frame Name: Bird

properties

wings 2

flies true

Class

Instances

Frame Name: Tweety

Class Bird

properties

wings 2

Frames

Considering the Cricket example ± Person, AdultMale, Batsman, Team are all classes. Sachin and India are entities. Both isa and instance relations have inverse attributes, which we call subclasses & all-instances. As a class is represents a set, there are 2 kinds of attributes that can be associated with it. Its own attributes & Attributes that are to be inherited by each element of the set.(* - This is how they are represented.)

EXAMPLE

- *Person*
  - *isa:* *Mammal*
  - *Cardinality:* *6000000000*
  - ** handed* *Right*

- *Adult-Male*
  - *isa:* *Person*
  - *Cardinality:* *2000000000*
  - **Height* *5-10*
- *Batsman*
  - *isa:* *Adult-Male*
  - *Cardinality:* *800*
  - **height* *6-1*
  - ** bats* *equal to handed*
  - **batting average* *.351*

Class

instance: class

isa: class

**cardinality:*

Team

instance: class

isa: class

Cardinality: {the no of teams that exist}

Teamsize: {each team has a size}

India

instance Team

**Coach*

**Team size 24*

**Players {sachin,dhoni……}*

Sachin

instance: India

instance: Batsman

Batting average .893

Frames as Sets and Instances

Sometimes, the difference between a set and an individual instance may not be clear . Example Team India is an instance of class of CricketTeams and can also be thought of as set of players. Now the problem is if were present Team India as a sub class of Cricket teams,then Indian players automatically become part of all the teams, which is not true. We have to do something to stop this. Instead we can make Team India a sub class of class called Cricket Players.To do this we need to differentiate between regular classes and metaclasses. Regular Classes are those whose elements are Individual entities whereas Metaclasses those are special classes whose elements are themselves classes.

Frames as Sets and Instances

The most basic meta class is the class CLASS. It represents the set of all classes. All classes are instances of it, either directly or through one of its subclasses. The class CLASS introduces the attribute cardinality, which is to be inherited by all instances of CLASS. Cardinality stands for number

Otherways of Relating Classes to Each Other

We have discussed that class1 can be a subset of class2.If class2 is a meta class then class1 can be an instance of class2.Another way is -mutually-disjoint-with relationship, which relates a class to one or more other classes that are guaranteed to have no elements in common with it.Another one is is-covered-by which relates a class to a set of subclasses, the union of which is equal to it.If a class is-covered-by a set S of mutually disjoint classes, then S is called a partition of the class.

Slots as Full-FledgedObjects(Frames)

To be able to represent these attributes of attributes, we need to describe attributes(slots) as frames.These frames will be organized into an isa hierarchy, just as any other frames are, and that hierarchy can then be used to support inheritance of values for attributes of slots. Now let us formalize what is a slot.A slot here is talked about as a relation. It maps from elements of its domain (the classes for which it makes sense) to elements of its range(its possible values).A relation is a set of ordered pairs. Thus it makes sense to say that relation R1 is a subset of another R2.In that case R1 is a specialization of R2.Since a slot is a set , the set of all slots, which we will call SLOT, is a meta class.Its instances are slots, which may have sub slots

SlotsValues as Objects

In the last section, we have used slots as explicit object that we could make assertions about. Here, we will discuss about using slot values´ as objects. Let's take the following example

John :

height : 72;tx (x.height > Bill.height)

Bill :

height : tx (x.height < John.height)

The only information we have here is that John is taller than Bill

Slot-Values as Objects

We need to make an statement about the value of a slot without knowing what the value is. To do that, we need to view the slot and its value as an object. We will expand our representation language to allow the value of slot to be stated as either or both of

- A value of the type required by the slot.
- A logical constraint on the value.This constraint may relate the slot's value to the values of other slots or to domain constrants.

John went to restaurant last night. He ordered large steak. When he paid for it , he noticed that he was running out of money. He hurried home since it had started to rain.

Scripts (strong slot and filler structure)

Did John eat dinner last night?

We can conclude that he had dinner from the natural sequence of events that occur in a restaurant.

Scripts (Schank & Abelson, 1975)

a script is a structure that describes a stereotyped sequence of events in a particular context

- Consists of a set of slots. Associated with each slot may be some information about what kinds of values it may contain as well as a default value to be used if no other information is available
  - closely resembles a frame , used to represent commonly occurring events such as going to the movies, but with additional information about the expected sequence of events and the goals/motivations of the actors involved
  - the elements of the script are represented using Conceptual Dependency relationships (as such, actions are reduced to conceptual primitives)

scripts

- There are 2 ways in which it may be useful to activate a script, they are
- For fleeting scripts(ones that are mentioned briefly and may be referred to again but are not central to the situation),it may be sufficient merely to store a pointer to the script so that it can be accessed later if necessary. This would be an appropriate strategy to take with respect to the restaurant script in the following story:
- Susan passed her favourite restaurant. On her way to the museum. She really enjoyed the Picasso exhibit.
- for nonfleeting scripts it is appropriate to activate the script fully and to attempt to fill in its slots with particular objects and people involved in the current situation
- The headers of a scripts (preconditions, props, roles, events etc.)can be used to activate a script.

Script Components

- Entry conditions : Conditions that must ,in general ,be satisfied before the events described in the script can occur.
- Results : Conditions that will, in general ,be true after the events described in the script have occurred.
- Props : Slots representing objects that are involved in the events described in the script. The presence of these objects can be inferred even if they are not mentioned explicitly .
- Roles : Slots representing people who are involved in the events described in the script. The presence of these people , too , can be inferred even if they are not mentioned explicitly. If specific individuals are mentioned ,they can be inserted into the appropriate slots.
- Track : The specific variation on a more general pattern that is represented by this particular script. Different tracks of the same script will share many but not all components.
- Scenes : The actual sequence of events that occur. The events are represented in conceptual dependency formalism

- EXAMPLE: restaurant script

describes: items usually found in a restaurant
people and their roles (e.g., chef, waiter, …)
preconditions and postconditions common scenes in
a restaurant: entering, ordering, eating, leaving

Script: RESTAURANT
Track: Coffee Shop
Props: Tables
Menu
F = Food
Check
Money

Roles: S = Customer
W = Waiter
C = Cook
M = Cashier
O = Owner

Scene 1: Entering

S PTRANS S into restaurant
S ATTEND eyes to tables
S MBUILD where to sit
S PTRANS S to table
S MOVE S to sitting position

Scene 2: Ordering

(Menu on table) (W brings menu)
S PTRANS menu to S

(S asks for menu)
S MTRANS signal to W
W PTRANS W to table
S MTRANS 'need menu' to W
W PTRANS W to menu

W PTRANS W to table
W ATRANS menu to S

S MTRANS food list to CP (S)
*S MBUILD choice of F
S MTRANS signal to W
W PTRANS W to table
S MTRANS I want F to W

W PTRANS W to C
W MTRANS (ATRANS F) to C

Entry conditions: S is hungry.
S has money.

Results: S has less money
O has more money
S is not hungry
S is pleased (optional)

C MTRANS 'no F' to W
W PTRANS W to S
W MTRANS 'no F' to S
(go back to *) or
(go to Scene 4 at no pay path)

C DO (prepare F script)
to Scene 3

Scene 3: Eating

C ATRANS F to W
W ATRANS F to S
S INGEST F

(Option: Return to Scene 2 to order more;
otherwise, go to Scene 4)

Scene 4: Exiting

S MTRANS to W
(W ATRANS check to S)

W MOVE (write check)
W PTRANS W to S
W ATRANS check to S
S ATRANS tip to W
S PTRANS S to M
S ATRANS money to M
S PTRANS S to out of restaurant

(No pay path)

Reasoning with scripts

Creation of a partially filled script named to meet the current situation .

Next , a known script which matches the current situation is recalled from memory .

The script name , or other keywords provide index values with which to search for the appropriate script.